

Function-Oriented Software Design

Commonly Made Errors

- Unbalanced DFDs
- Forgetting to mention names of the data flows
- Unrepresented functions or data
- External entities appearing at higher level DFDs
- Trying to represent control aspects
- Context diagram having more than one bubble
- A bubble decomposed into too many bubbles in the next level
- Terminating decomposition too early
- Nouns used in naming bubbles

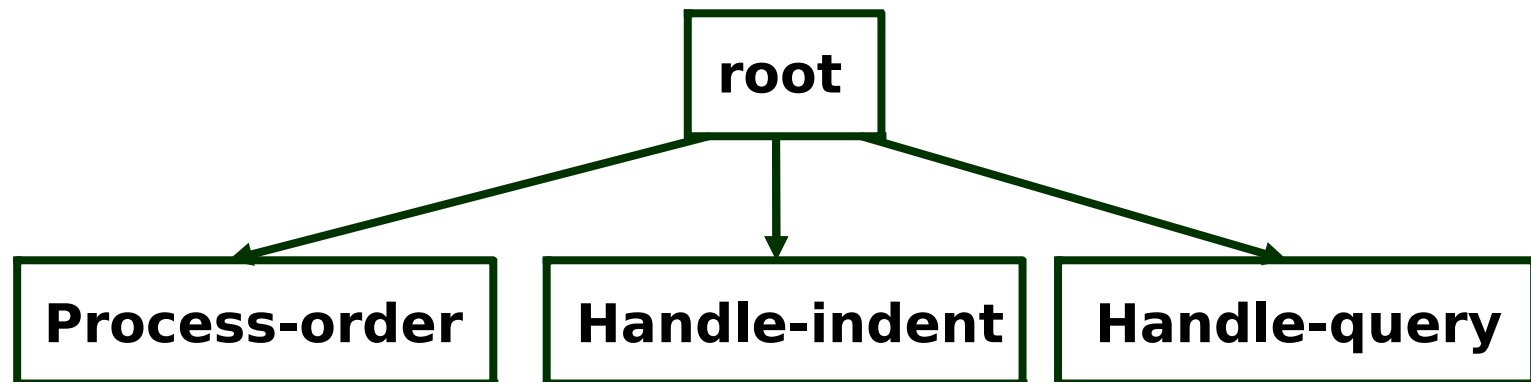
Structured Design

- Aim of structured design
 - Transform results of structured analysis (i.e., a DFD representation) into a structure chart.
- A structure chart represents the system modules.
 - Various modules making up the system,
 - Module dependency (i.e. which module calls which other modules),
 - Parameters passed among different modules.
- Structure chart representation
 - Procedural aspects (e.g, how a particular functionality is achieved) are not represented.
- Main focus of a structure chart:
 - Define the module structure of a software,
 - Interaction among different modules,
 - Easily implementable using programming languages.

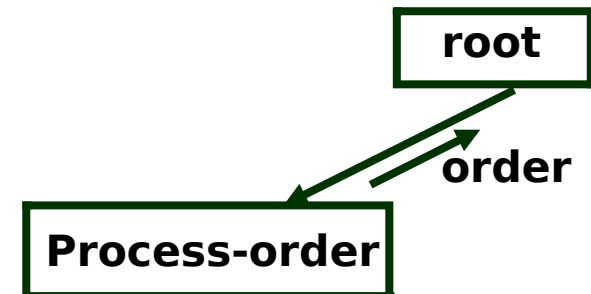
Basic Building Blocks of Structure Chart

- Rectangular box:
 - A rectangular box represents a module.
 - Annotated with the name of the module it represents.
- An arrow between two modules implies:
 - During execution control is passed from one module to the other in the direction of the arrow.

Process-order



- Data flow arrows represent:
 - Data passing from module to another in the direction of arrow.

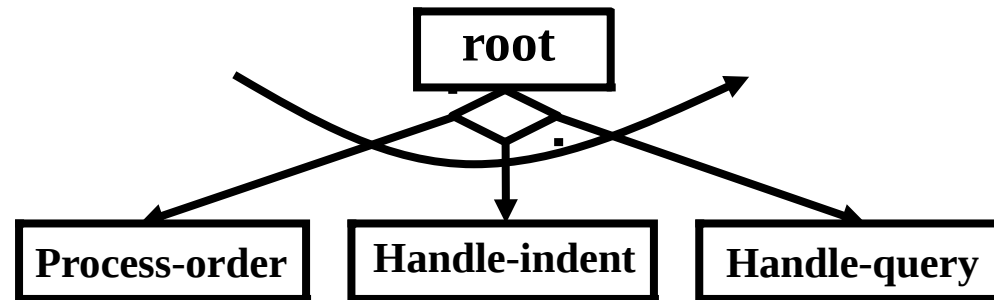


Basic Building Blocks of Structure Chart

- Library modules represent frequently called modules:
 - A rectangle with double side edges.
 - Simplifies drawing when a module is called by several modules.



- Selection (using a diamond symbol)
 - One module of several modules connected to the diamond symbol is invoked depending on some condition.



- Repetition
 - A loop around control flow arrows denotes that the concerned modules are invoked repeatedly.

Structure Chart

- There is only one module at the top - the **root module**.
- There is at most one control relationship between any two modules:
 - if module A invokes module B, module B cannot invoke module A.
- The main reason behind this restriction – **consideration of modules in a structure chart being arranged in layers or levels following the standard abstraction principle.**
- Principle of abstraction
 - does not allow lower-level modules to invoke higher-level modules:
 - But, two higher-level modules can invoke the same lower-level module.

Bad Design

